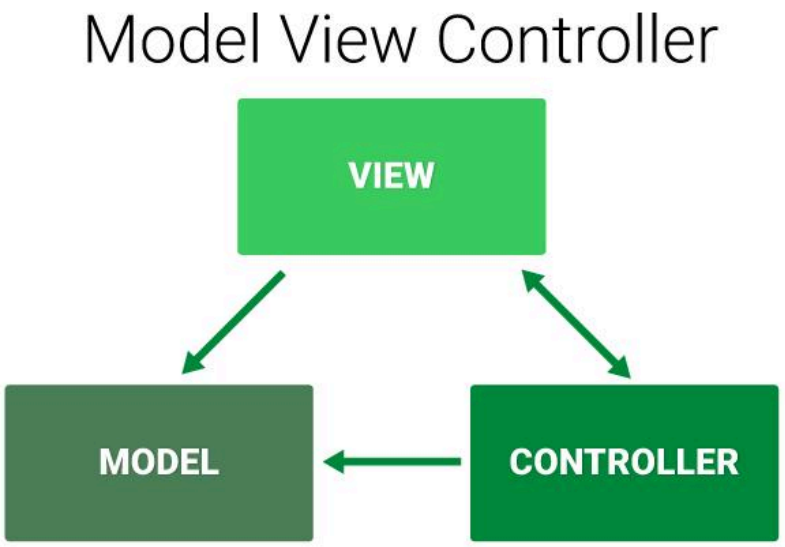
FSWD — Important Questions for MSE 1

UNIT-1

1. Explain MVC Architecture ? How they are related to MERN?

MVC (Model–View–Controller) is an architectural pattern used to divide a web application into three interconnected parts. It follows **Separation of Concerns (SoC)**, making the application more organized and manageable.



Components of MVC

1. Model

- Manages the **business logic and data** of the application.
- Determines how data is **stored, created, and modified**.

2. View

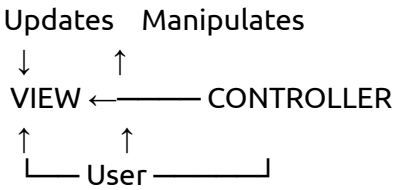
- Represents the **visual/UI part** of the application.
- Displays data and information to the user.

3. Controller

- Acts as an intermediary between the **User, Model and View**.
- Interprets user-generated events and gives commands to the Model and View for appropriate updates.

Working of MVC

MODEL
↙ ↘



The **User interacts with the View**. The Controller receives the user's actions and communicates with the Model. The Model processes the data and the View displays the updated information.

Relationship between MVC and MERN

MERN consists of **MongoDB, Express.js, React.js and Node.js**, with JavaScript connecting the different components.

They can be related to MVC as follows:

MVC Component	MERN Technology	Role
Model	MongoDB	Stores and manages data
Controller	Express.js + Node.js	Handles requests and backend logic
View	React.js	Creates the user interface

React is particularly related to the View: the PPT states that React is a JavaScript library used for creating views rendered in HTML and can be used as the **V (View) in MVC**. However, React itself does **not enforce the MVC pattern**.

In short

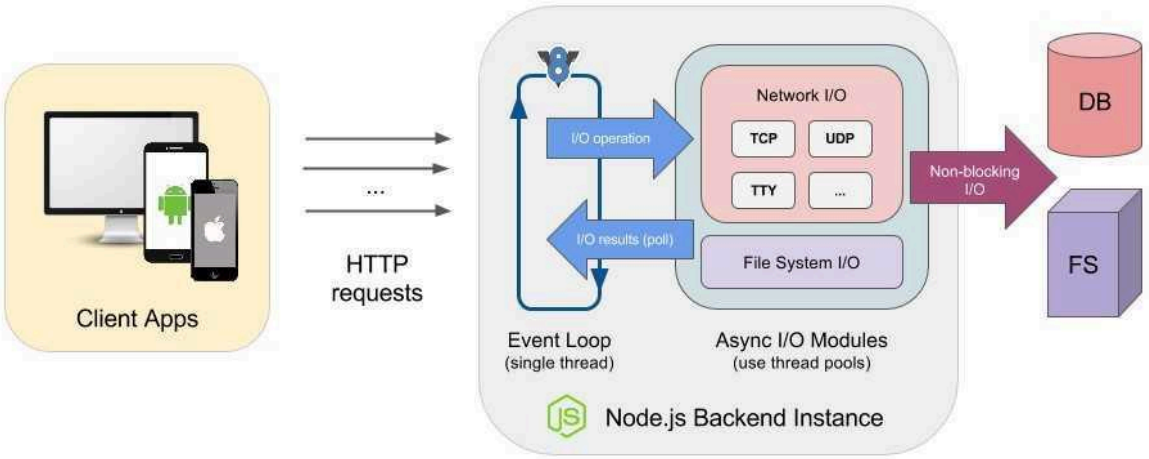
```
MERN                        MVC

MongoDB          ————————→  Model
Express + Node   ————————→  Controller
React            ————————→  View
```

Conclusion: MVC is an **architectural pattern**, whereas MERN is a **technology stack**. A MERN application can be structured using the MVC pattern to achieve separation of concerns and a consistent, manageable application structure.

2. Explain Features of Node JS?

Node.js is a JavaScript runtime environment that allows JavaScript to run **outside the browser**, mainly on the server. It is built on Google's **V8 JavaScript engine** and is used for backend development, web servers, and REST APIs.



Features of Node.js

1. Asynchronous and Non-Blocking I/O
Node.js does not wait for an input/output operation to finish. It continues executing other tasks and handles the result when the operation is completed. This improves responsiveness.

2. Event-Driven Architecture
Node.js uses an **event-driven model**, where events such as file operations, database queries, and network requests trigger callback functions.

3. Single-Threaded Event Loop
Node.js uses a **single-threaded event loop** to efficiently handle multiple requests instead of creating a separate thread for every request. This makes it suitable for applications with many concurrent users.

4. High Performance
Node.js is built on Google's **V8 JavaScript engine**, which executes JavaScript efficiently. Its event-driven and non-blocking architecture provides fast performance.

5. Scalable
Because of its non-blocking and event-driven architecture, Node.js can efficiently handle a large number of concurrent requests, making it suitable for scalable web applications.

6. JavaScript on the Server
Node.js allows developers to use **JavaScript for backend development**, including web servers and REST APIs. Thus, JavaScript can be used for both frontend and backend development.

7. Module System
Node.js uses the **CommonJS module system** to organize code into separate modules.

Modules can be imported using require(). It also provides built-in modules such as fs, http, os, and path.

8. npm Ecosystem
Node.js comes with **npm (Node Package Manager)**, which allows developers to install and manage reusable third-party packages such as Express, React, and Mongoose.

9. Cross-Platform
Node.js applications can run on different operating systems such as **Windows, Linux, and macOS**, making it suitable for developing portable applications.

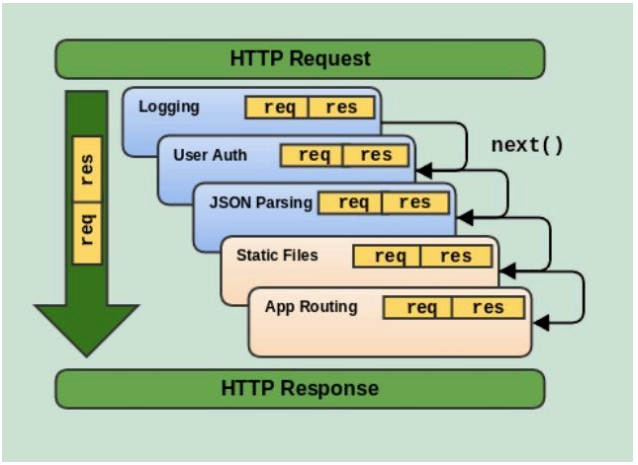
Simple Diagram

Conclusion

Node.js is a **fast, asynchronous, event-driven and non-blocking JavaScript runtime**. Its **V8 engine, event loop, module system, scalability and large npm ecosystem** make it well suited for modern web servers, APIs and backend applications.

3. Express - server.js ? how they are going to communicate with static.js (5 lines of code and explain)

In the PPT, Express is used to **serve static files**. The express.static() middleware looks for requested files inside a specified directory and sends the file to the browser.



5 lines of code — server.js

JavaScript

```
const express = require('express');
const app = express();
app.use(express.static('public'));
app.listen(3000, () => {
    console.log('Server started');
});
```

These are essentially the **same 5 lines used in Ma'am's PPT** for the Express server.

How does it communicate with static.js?

Suppose the project is:

```
project/
|
├── server.js
└── public/
    ├── index.html
    └── static.js
```

Inside index.html:

<script src="/static.js"></script>

Explanation

1. require('express') → imports the **Express module**.

2. express() → creates the Express application.
3. express.static('public') → tells Express to **serve files from the public folder**.
4. app.listen(3000) → starts the server on **port 3000**.
5. When the browser requests /static.js, Express searches the public folder, finds static.js, and **sends its contents to the browser**.

In one flow

Browser
  |
  | GET /static.js
  ↓
server.js (Express)
  |
  | express.static('public')
  ↓
public/static.js
  |
  ↓
Browser receives & executes JavaScript

Exam conclusion: Express does not directly "communicate" with static.js by calling it. Instead, **express.static() serves static.js as a static file when the browser requests it.**

4. Explain features of MongoDB.

MongoDB is a **NoSQL, document-oriented database** used in the MERN stack. It stores data in flexible, JSON-like documents instead of traditional rows and columns.

Features of MongoDB

1. NoSQL Database
MongoDB is a **non-relational database**. It does not store data in traditional tables with rows and columns. It uses flexible documents instead.

2. Document-Oriented
Data is stored as **documents**, and multiple documents are grouped into **collections**.

```
Database
   ↓
Collection
   ↓
Document
   ↓
{ name: "Rahul", age: 22 }
```

Each document has a unique identifier (_id).

3. Schema-Less / Flexible Schema
MongoDB does not require a predefined schema. Documents in the same collection can have different fields, and new fields can be added easily.

4. JSON-Like Data Format
MongoDB stores data in a JSON-like structure, which is very similar to JavaScript objects. Internally, MongoDB stores documents in **BSON (Binary JSON)**.

Example:

```javascript
{
  name: "Rahul",
  course: "MCA",
  age: 22
}
```

5. Easy Integration with JavaScript
Since MongoDB uses JSON-like documents, its data structure closely matches JavaScript objects used in **Node.js and React**, making development easier.

6. Supports CRUD Operations
MongoDB supports the four basic database operations:

- **C** – Create
- **R** – Read
- **U** – Update
- **D** – Delete

These operations can be performed using JSON-like syntax.

7. Indexing
MongoDB supports indexing, including indexes on **nested fields**. Indexes help MongoDB find data faster.

8. Horizontal Scalability
MongoDB can scale horizontally by **distributing data across multiple servers**, making it suitable for large-scale applications.

9. Supports Nested Data
A document can contain objects and arrays inside it. This allows related data to be stored together.

```javascript
{
  name: "Rahul",
  address: {
    city: "Mangalore",
    state: "Karnataka"
  }
}
```

MongoDB can also index these nested fields.

Conclusion

MongoDB is a **flexible, document-oriented NoSQL database** that supports JSON-like data, CRUD operations, indexing, nested documents, and horizontal scalability. Because it works naturally with JavaScript, it is widely used as the **database component of the MERN stack**.

5. Explain any 2 tools and Libraries.

Tools and libraries make web development **easier and faster** by providing ready-made features, reducing development time, and improving code quality.

1. React Router

- Used for **navigation** between different React components/pages.
- Keeps the browser URL synchronized with the displayed component.
- Supports **Back and Forward** buttons.
- Enables **Single Page Application (SPA)** navigation without reloading the whole page.

2. React-Bootstrap

- A React library based on the **Bootstrap CSS framework**.
- Provides ready-made UI components.
- Helps create **responsive and attractive** web applications.
- Makes UI development easier.

3. Webpack

- A **module bundler** used in JavaScript and React applications.
- Combines JavaScript, CSS, images and other resources into optimized bundles.
- Works with **Babel** to convert JSX into browser-understandable JavaScript.
- Simplifies the build process and improves performance.

4. Redux

- A **state management library**.
- Stores shared application data in a centralized place.
- Useful for **large and complex applications** where managing state becomes difficult.

5. Mongoose

- An **ODM (Object Document Mapper)** for MongoDB.
- Provides an abstraction between Node.js applications and MongoDB.
- Supports **schema definition and data validation**.
- Simplifies database operations.

6. Other Libraries

- **body-parser** – Parses incoming HTTP request data and converts JSON/form data into JavaScript objects.
- **ESLint** – Checks JavaScript code for errors and enforces coding standards.
- **react-select** – Provides customizable dropdown components.
- **Jest** – A testing library used for testing React applications.

Simple Summary

```
Tools & Libraries
       |
       ├── React Router → Navigation
       ├── React-Bootstrap → UI
       ├── Webpack → Bundling
       ├── Redux → State Management
       ├── Mongoose → MongoDB
       ├── ESLint → Code Quality
       └── Jest → Testing
```

Conclusion: These tools and libraries extend the MERN stack and help developers build applications that are **faster, organized, maintainable, responsive, and easier to develop**.

UNIT-2

Chapter 1

1. Automate -> commands , change in package.json , inside script tag (start watch)

In Node.js, we can **automate frequently used commands** by adding them inside the **scripts** section of package.json. This avoids typing long commands repeatedly.

Example: package.json

```JSON
{
  "scripts": {
    "start": "node server.js",
    "watch": "nodemon server.js"
  }
}
```

Explanation

1. start script

"start": "node server.js"

When we run:

npm start

Node.js automatically executes:

node server.js

So we don't need to type the complete command every time.

---

2. watch script

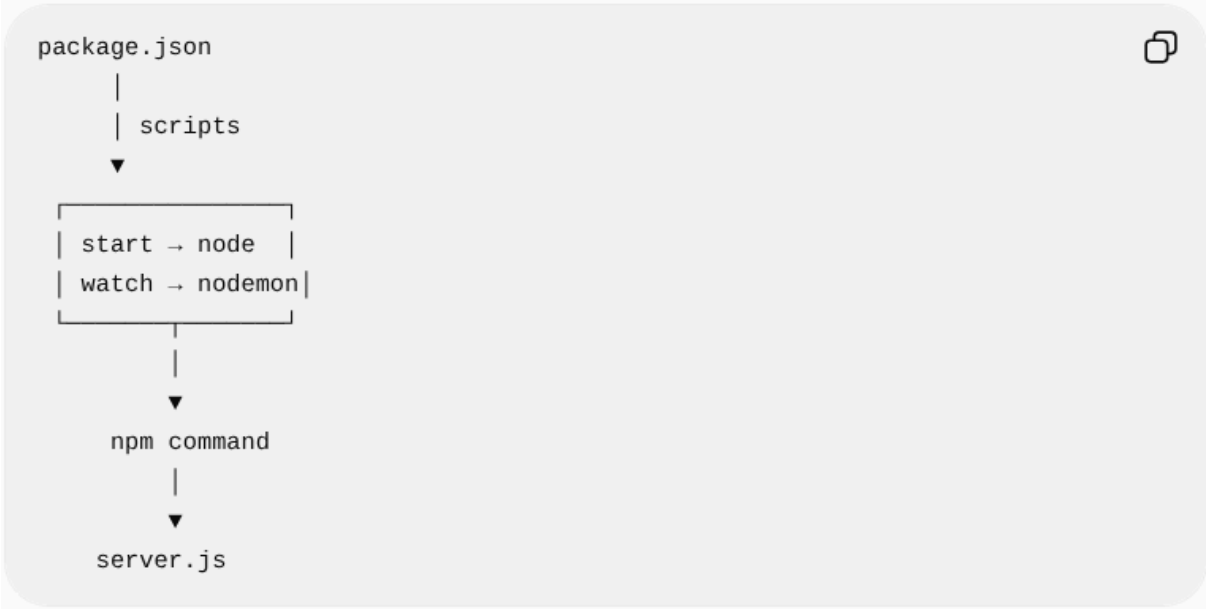
"watch": "nodemon server.js"

When we run:

npm run watch

Nodemon starts server.js and automatically restarts the server whenever we make changes to the code.

How it works



Important commands

npm start

→ Runs the **start** script.

npm run watch

→ Runs the **watch** script.

In short: The scripts section of package.json is used to **automate commands**, while start runs the server normally and watch runs it using Nodemon for automatic restarting.

2. Command npm init -y and npm install express , npm start. Explain purpose, When executed where executed, where will it make changes in project

These are **npm commands** used while setting up and running a Node.js/Express project.

```
C:\Users\Arjun Samrat>cd Desktop

C:\Users\Arjun Samrat\Desktop>mkdir my-node-project

C:\Users\Arjun Samrat\Desktop>cd my-node-project

C:\Users\Arjun Samrat\Desktop\my-node-project>npm init -y
Wrote to C:\Users\Arjun Samrat\Desktop\my-node-project\package.json:

{
  "name": "my-node-project",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}



C:\Users\Arjun Samrat\Desktop\my-node-project>npm install express

added 64 packages, and audited 65 packages in 5s

12 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

1. npm init -y

Purpose: Creates a new Node.js project and generates a **package.json** file.

When executed: At the **beginning of a new Node.js project**.

Where executed: In the **project's root folder** using the terminal.

npm init -y

What changes in the project?

project/
└── package.json ← CREATED

The package.json contains project information such as the project name, version, and scripts.

-y automatically accepts the default options instead of asking questions.

---

2. npm install express

Purpose: Downloads and installs the **Express.js package** so that we can use Express in our Node.js application.

When executed: After creating the Node.js project, when we need Express.

Where executed: In the **project's root folder**.

npm install express

What changes in the project?

```
project/
├── package.json      ← Express added to dependencies
├── package-lock.json ← Created/updated
└── node_modules/     ← Express and its dependencies installed
```

For example, package.json will contain something similar to:

```
"dependencies": {
  "express": "^5.x.x"
}
```

So, **npm install express makes changes mainly to package.json, package-lock.json, and node_modules.**

---

3. npm start

Purpose: Runs the command specified by the **start script** in package.json.

For example:

```
"scripts": {
  "start": "node server.js"
}
```

Then:

npm start

executes:

node server.js

When executed: Whenever you want to **start/run your application**.

Where executed: In the **project's root folder**, where package.json is located.

What changes in the project?

Normally, **npm start does not modify your project files**. It simply reads package.json and executes the specified command.

⭐ Easy exam table

Command	Purpose	When	Project changes
npm init -y	Creates Node project	Project setup	Creates package.json
npm install express	Installs Express	Before using Express	Updates package.json, package-lock.json, creates node_modules
npm start	Starts application	When running project	Normally **no file changes**

All three commands are normally executed from the project's root directory in the terminal.

3. watch command - nodemon

Nodemon is a tool used during Node.js development to **automatically restart the server whenever changes are made to the source code**.

Add it to package.json

```json
{
  "scripts": {
    "start": "node server.js",
    "watch": "nodemon server.js"
  }
}
```

Command

npm run watch

This executes:

nodemon server.js

How it works

```
You modify server.js
        ↓
    Nodemon detects change
        ↓
   Server automatically
       restarts
        ↓
 Updated application runs
```

Why use Nodemon?

Without Nodemon:

Change code → Stop server → Start server again

With Nodemon:

Change code → Nodemon automatically restarts

Important: watch is **not a built-in npm command**. It is a **custom script name** defined inside package.json. Nodemon is the tool that watches the files and restarts the server.

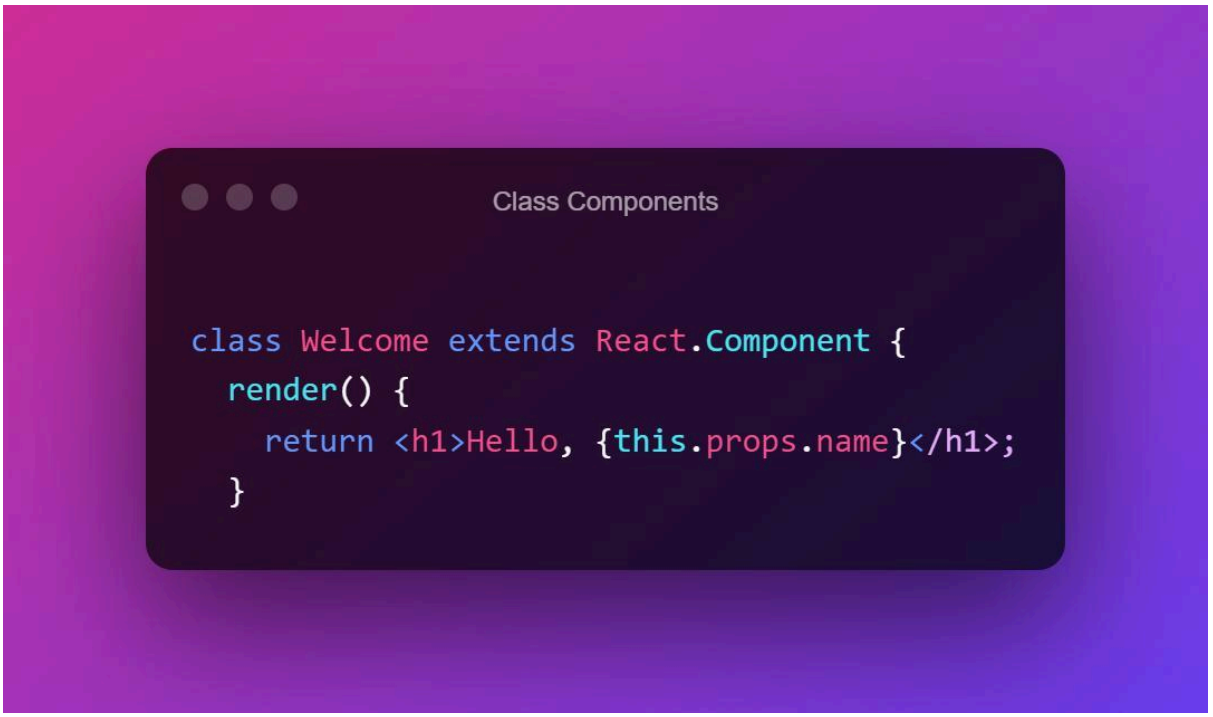
Exam point:

> The watch script is used to run the Node.js application with Nodemon, which automatically restarts the server whenever changes are detected in the source files.

Chapter 2

4. class components ? How to create ? give syntax and example and explain?

A **Class Component** is a React component created using a JavaScript class. It is created by **extending** **React.Component** and must contain a **render() method**, which returns the UI to be displayed.



Syntax

JavaScript

```
class ComponentName extends React.Component {
    render() {
        return (
            <div>
                Content
            </div>
        );
    }
}
```

Example

JavaScript

```
class HelloWorld extends React.Component {
    render() {
        return (
            <div>
                <h1>Hello World!</h1>
            </div>
        );
    }
}

const element = <HelloWorld />;
```

Explanation

1. **class HelloWorld** → Creates a new React class component named HelloWorld.
2. **extends React.Component** → Inherits React's component functionality. All custom class components are derived from React.Component.
3. **render()** → React calls this method when it needs to display the component.
4. **return()** → Returns the JSX/UI that should appear on the screen.
5. **<HelloWorld />** → Creates an instance of the HelloWorld component so it can be rendered.

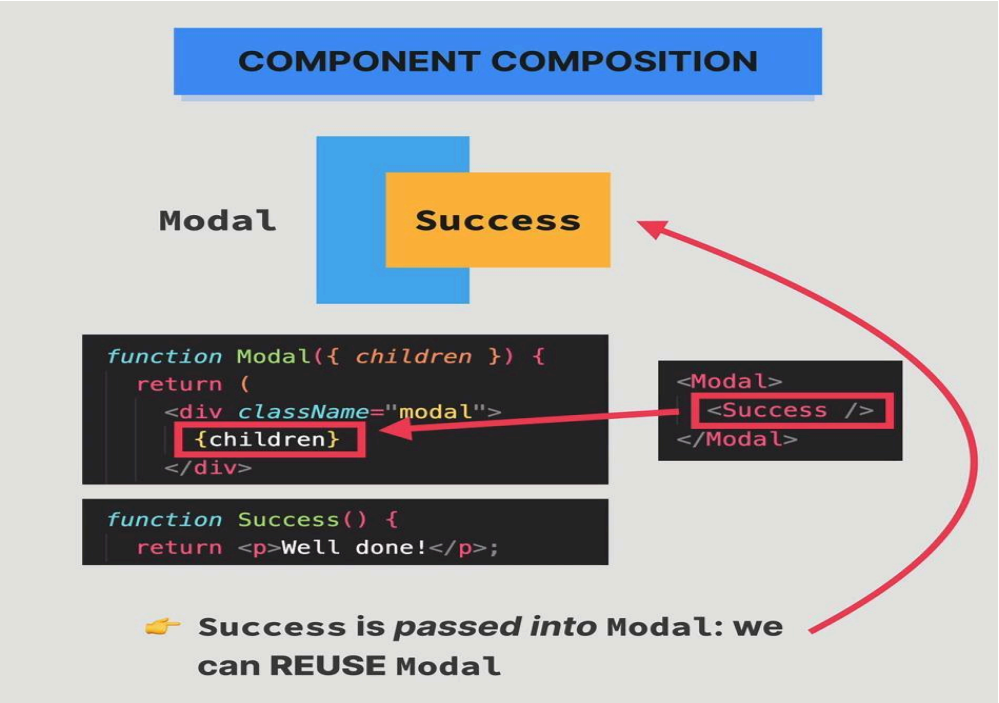
Simple flow

```
class HelloWorld
       ↓
extends React.Component
       ↓
     render()
       ↓
   returns JSX
       ↓
   Displayed in UI
```

In short: A class component is a reusable React component created by extending React.Component. The render() method defines what UI the component displays.

5. Explain component composition

Component Composition is a React technique where we **combine smaller, reusable components to create a larger component**. Instead of putting all the UI and logic in one component, we divide it into smaller components and compose them together.



Example

JavaScript

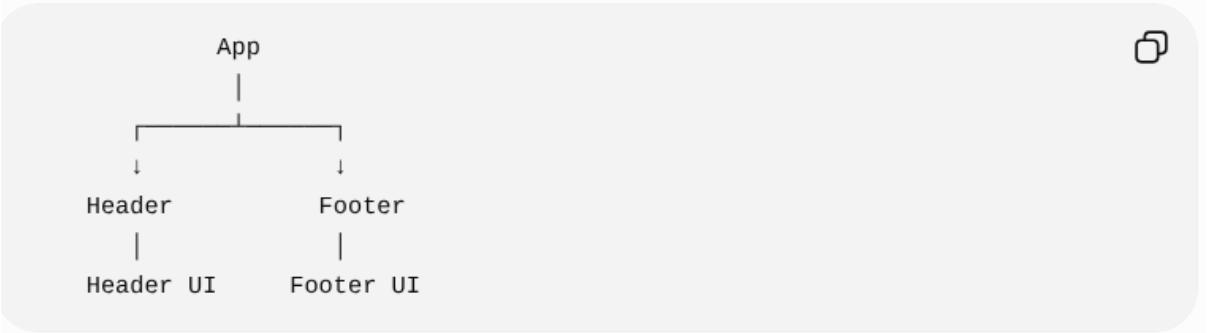
```javascript
function Header() {
    return <h1>My Website</h1>;
}

function Footer() {
    return <p>Copyright 2026</p>;
}

function App() {
    return (
        <div>
            <Header />
            <h2>Welcome!</h2>
            <Footer />
        </div>
    );
}
```

Explanation

- Header is a **small reusable component** for the website header.
- Footer is another **reusable component**.
- App is the **parent component** that combines Header and Footer.
- <Header /> and <Footer /> are used inside App to compose the complete UI.



Advantages

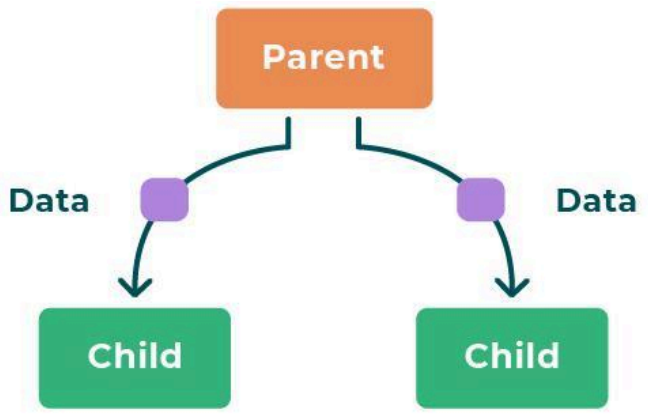
1. **Reusability** – The same component can be used in multiple places.
2. **Maintainability** – Smaller components are easier to understand and modify.
3. **Separation of concerns** – Each component can handle a specific part of the UI.
4. **Simplifies complex applications** – Large interfaces can be divided into manageable components.

In short:

> **Component Composition means building a complex React UI by combining smaller, independent and reusable components together.**

6. Explain passing data with properties.

Props (properties) are used in React to **pass data from a parent component to a child component**. Props are read-only, meaning the child component should not modify them.



Example

JavaScript

```
function Student(props) {
    return <h2>Hello, {props.name}</h2>;
}

function App() {
    return (
        <div>
            <Student name="Shashi" />
        </div>
    );
}
```

Explanation

1. Parent component

<Student name="Shashi" />

App is the parent component. It passes the value **"Shashi"** to Student using the name property.

2. Child component

JavaScript

```
function Student(props) {
    return <h2>Hello, {props.name}</h2>;
}
```

Student receives the props through the props parameter and accesses the value using:

props.name

3. Output

Hello, Shashi

Data Flow

```
Parent (App)
     |
     | name="Shashi"
     ↓
Child (Student)
     |
     | props.name
     ↓
Hello, Shashi
```

Important Points

- Props are used for **communication between components**.
- Data normally flows **from parent → child**.
- Props can contain **strings, numbers, arrays, objects, and functions**.
- Props are **read-only**; the child should not directly modify them.

In short:

> **Props allow a parent component to pass data to a child component, making React components reusable and dynamic.**